Classroom Challenge

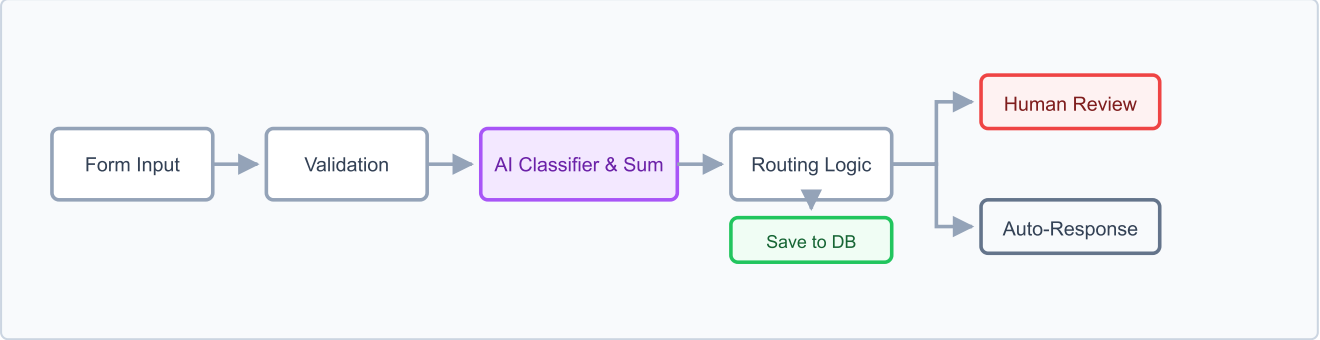
Building an AI-Powered Student Support Workflow

This challenge integrates data validation, AI classification, summarization, and conditional routing into a robust student support pipeline. The goal is to automate the handling of general academic queries while ensuring sensitive or urgent matters escalate to human advisors immediately.

Project Requirements

- **1. Ingestion:** Accept a student query through a standard form payload.
- **2. Validation:** Ensure the submitted data contains a valid student ID and readable text.
- **3. Categorization:** Use AI to classify the query (e.g., General, Urgent, Unclear).
- **4. Summarization:** Generate a short summary of the query for quick advisor review.
- **5. Routing:** Direct the ticket to the appropriate channel based on its category.
- **6. Storage:** Save the original query, summary, and routing result in a database.
- **7. Escalation:** Flag urgent or unclear queries for mandatory human oversight.
- **8. Error Handling & Testing:** Record pipeline failures gracefully and validate with sample inputs.

Workflow Architecture



Phase 1: Ingestion, Validation & AI Processing

The first block of our implementation handles receiving the form data, ensuring it is structurally sound, and passing it to the simulated AI engine for classification and summarization.

```
import uuid

def process_student_query(payload):
    # 1 & 2. Ingestion & Validation
    student_id = payload.get("student_id")
    query_text = payload.get("query")

    if not student_id or not query_text or len(query_text.strip()) < 5:
        return record_failure("Validation Error: Missing ID or invalid query length.")

    try:
        # 3 & 4. AI Categorization and Summarization (Mocked)
        ai_response = call_ai_assistant(query_text)
        category = ai_response["category"]
        summary = ai_response["summary"]
    except Exception as e:
        return record_failure(f"AI Service Failure: {str(e)}")

    # Proceed to Phase 2: Routing...
    return route_query(student_id, query_text, category, summary)

def call_ai_assistant(text):
    """Mocks an LLM returning structured JSON with a category and summary."""
    text_lower = text.lower()
    summary = f"Student is inquiring about: {text[:30]}..."

    if "mental health" in text_lower or "harassment" in text_lower:
        return {"category": "Urgent", "summary": "Sensitive student safety issue."}
    elif "schedule" in text_lower or "drop class" in text_lower:
        return {"category": "General", "summary": summary}
    else:
        return {"category": "Unclear", "summary": "Topic not recognized."}
```

> **Defensive Programming**
>
> Notice the `try/except` block wrapping the AI service call. External APIs are the most common failure point in automation pipelines. Wrapping this call ensures that if the LLM times out or returns malformed JSON, the system captures the failure instead of crashing silently, satisfying Requirement 8.

Phase 2: Routing, Storage & Testing

Once classified, conditional logic directs the payload. All outcomes are saved to a simulated database.

```
def route_query(student_id, original_text, category, summary):
    ticket_id = str(uuid.uuid4())[:8]
    action_taken = ""

    # 5 & 7. Routing and Escalation Logic
    if category == "Urgent" or category == "Unclear":
        action_taken = "Flagged for Human Review"
    elif category == "General":
        action_taken = "Sent Automated Knowledge Base Link"
    else:
        action_taken = "Fallback routing"

    # 6. Save to Database
    db_record = {
        "ticket_id": ticket_id,
        "student_id": student_id,
        "category": category,
        "summary": summary,
        "action": action_taken
    }
    mock_database_save(db_record)
    return db_record

def record_failure(reason):
    print(f"[SYSTEM ALERT] Workflow failed: {reason}")
    return {"status": "Error", "reason": reason}

def mock_database_save(record):
    print(f"Saved to DB: {record['ticket_id']} - {record['action']}")
```

8. Testing with Sample Inputs

Scenario	Input Payload	Expected Category	Expected Routing Action
Valid General	`{"student_id": "992", "query": "How do I drop a class?"}`	General	Sent Automated Knowledge Base Link
Valid Urgent	`{"student_id": "118", "query": "Experiencing severe harassment"}`	Urgent	Flagged for Human Review
Vague Input	`{"student_id": "404", "query": "I don't know what to do."}`	Unclear	Flagged for Human Review
Invalid Data	`{"student_id": "", "query": "Hi"}`	*None (Fails Validation)*	Error: Missing ID or invalid length